FINVAULT – VÍ ĐIỆN TỬ

NHIỆM VỤ CHI TIẾT TUẦN 2

PIN giao dịch · transactionToken · Hạn mức theo KYC · Chuyển tiền P2P · Thông báo realtime

Phân công cụ thể cho từng thành viên và giải thích các khái niệm quan trọng

Hạng mục	Nội dung
Nhóm	Đạt · Khoa · Hữu · Nguyên (4 người × 5 ngày = 20 ngày công)
Thời gian	Tuần 2 (Sprint 2), 5 ngày làm việc; ngày 5 là ngày đóng băng (chỉ sửa lỗi)
Nền tảng demo	Web (React/Vite). Mobile chỉ làm đăng nhập nếu còn sức (quyết định M20)
Nguồn	Bản "Đánh giá Kế hoạch Tuần 2 và bản kiến trúc, phân công bổ sung" (18/10/2026) và mã nguồn backend (repomix-output.xml)
Phiên bản	1.0 – 19/10/2026

Mục lục

Nhấn chuột phải → Update Field (hoặc F9) để cập nhật mục lục.

0. Cách đọc tài liệu này

Tài liệu viết cho cả nhóm, kể cả người chưa từng làm hệ thống tài chính. Mỗi người chỉ cần đọc **mục 1–3** (bức tranh chung), **chương của chính mình** (mục 4–7), và **mục 9 – Khái niệm** cho các thuật ngữ mình chưa rõ. Mục 8 (hợp đồng API, sự kiện) là "luật chung" khi hai người nối code với nhau.

Ký hiệu	Ý nghĩa
Đ7, K8, H7, N8…	Mã việc. Chữ cái đầu = người làm (Đ = Đạt, K = Khoa, H = Hữu, N = Nguyên). B1–B7 là việc bù còn nợ của Tuần 1. T1 là tự động hóa nghiệm thu.
Must	Bắt buộc. Thiếu là Tuần 2 không đạt. Không bao giờ cắt.
Should	Nên làm. Làm khi Must đã xanh. Có thể dời sang Tuần 3.
Could	Làm nếu dư công. Cắt đầu tiên khi trễ.
Ngày công	1,0 = một người làm tập trung một ngày. 0,5 = nửa ngày.
Xong khi	Điều kiện kiểm được (test hoặc thao tác demo). Chưa thỏa "Xong khi" thì chưa được báo xong.
M8…M25	Quyết định kỹ thuật đã chốt (xem mục 10). Ai không đồng ý thì bình luận trước 12:00 ngày 1.
P0-x, P1-x	Lỗ hổng tìm thấy trong code. P0 = chặn tiến độ, phải sửa trong Tuần 2.
S1…S16	Bài kiểm thử bảo mật / đúng đắn mới (mục 11).

1. Tổng quan Tuần 2

1.1 Mục tiêu một câu

Cuối Tuần 2, trên Web, người dùng A chuyển tiền cho B như sau:

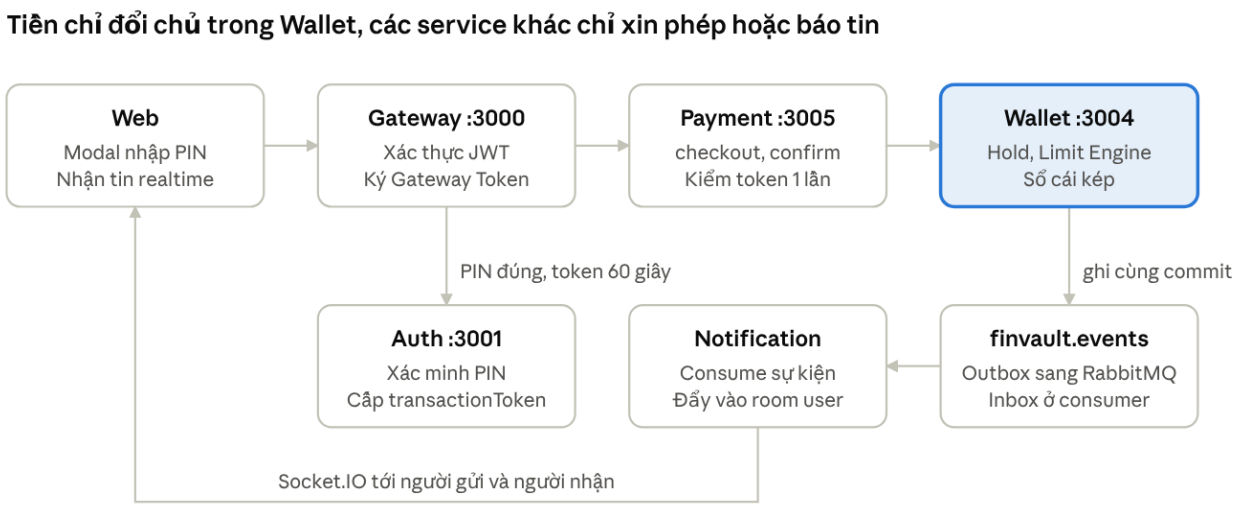
- Tìm B bằng email hoặc số điện thoại, thấy tên đã che (ví dụ "Nguyễn V. B.").
- Nhập số tiền, xem báo giá (phí 0đ), xác nhận.
- Nhập **PIN 6 số**; sai 5 lần thì bị khóa 30 phút.
- Hệ thống kiểm **hạn mức theo cấp KYC**; vượt thì báo rõ còn được chuyển bao nhiêu.
- Tiền chuyển đúng một lần (bấm đúp không tạo hai giao dịch); B thấy **thông báo trong ≤ 3 giây**.
- Admin duyệt CCCD hai mặt → người dùng lên Cấp 1 → được chuyển số tiền lớn hơn.

1.2 Định nghĩa hoàn thành (DoD) của Tuần 2

- 5 kịch bản gốc **A3** (khóa PIN), **A4** (token dùng lại), **A9** (hạn mức Cấp 0), **R1** (thông báo ≤ 3 giây), **K1** (duyệt KYC nâng hạn mức) đều xanh.
- 16 bài kiểm thử mới **S1–S16** đều xanh, trong **một lần chạy** trên môi trường `docker compose` sạch.
- Sau khi chạy, **tổng số dư toàn hệ thống bằng 0** (kiểm bằng `checkSystemConservation`).
- Không còn secret mặc định gõ cứng trong repo (ví dụ `default_internal_secret_key_123456`).

1.3 Kiến trúc và luồng chuyển tiền P2P

Kiến trúc giữ nguyên 8 service hiện có, **không thêm Redis**. Tuần 2 chỉ thêm ba thứ: **PIN + transactionToken** ở Auth, **Limit Engine** ở Wallet, và **exchange `finvault.events`** nối mọi service với Notification. Nguyên tắc vàng: **tiền chỉ đổi chủ trong Wallet**, các service khác chỉ "xin phép" hoặc "báo tin".



Hình 1. Luồng P2P Tuần 2 – 6 service, 1 exchange sự kiện.

#	Bước	Ai làm	Chuyện gì xảy ra
1	Tra người nhận	Hữu (H11)	Web gọi `GET /api/users/lookup?q=` theo email/SĐT, nhận `authUserId` và `maskedName`.
2	Báo giá	Khoa (K9)	`POST /api/payments/p2p/quote` trả số tiền, phí 0đ, hạn 60 giây để màn hình xác nhận hiển thị.
3	Tạo giao dịch	Khoa (K8, K9)	`POST /api/payments/p2p/checkout` kèm header `Idempotency-Key`. Payment tạo bản ghi PENDING, gọi Wallet **hold** có `destinationUserId`. Wallet kiểm và cộng hạn mức **trong cùng transaction** với hold; vượt → 422 `LIMIT_EXCEEDED`. Hold sống 5 phút.
4	Nhập PIN	Đạt (Đ7, Đ10)	Web mở Modal PIN, gọi `POST /api/auth/pin/verify` kèm `paymentId`, `amount`. Đúng PIN → nhận **transactionToken** sống 60 giây; sai → tăng bộ đếm khóa.
5	Xác nhận	Khoa + Đạt (K9, Đ8b)	`POST /api/payments/:id/confirm` kèm header `X-Transaction-Token`. Middleware kiểm chữ ký, claims khớp số tiền và paymentId, ghi `jti` vào bảng UNIQUE (token chỉ dùng 1 lần). Payment →

#	Bước	Ai làm	Chuyện gì xảy ra
			PROCESSING.
6	Chuyển tiền	Khoa (K9)	Payment gọi Wallet **capture**: bút toán kép trong 1 transaction. Thành công → SUCCESS. Lỗi → FAILED + **release** hold, hoàn hạn mức. **Không còn nhánh bù tiền bằng credit.**
7	Phát sự kiện	Nguyên (N11)	Wallet, Payment ghi **Outbox** cùng transaction với dữ liệu; publisher đẩy vào `finvault.events` khi broker xác nhận.
8	Thông báo	Nguyên (N8)	Notification consume qua **Inbox** chống trùng, lưu thông báo, đẩy Socket.IO vào room `user:<id>` của **cả người gửi lẫn người nhận**.

Hai ranh giới an ninh phải giữ

- Gateway **chặn người dùng gọi trực tiếp** các route tiền của Wallet (hold, capture, release, transfer, credit, debit) – quyết định M23.
- Mọi route chạm tiền ngoài Payment **chỉ nhận lời gọi nội bộ hoặc Admin**.

Lý do: hiện tại chủ ví gọi được `POST /api/wallets/<id của mình>/transfer` bằng access token thường → bỏ qua PIN và hạn mức (lỗ hổng P0-1).

1.4 Bảy lỗ hổng P0 phải sửa (lý do có các việc trong tuần này)

Mã	Vấn đề (nói đơn giản)	Bằng chứng trong code	Ai sửa, trong việc nào
P0-1	Chủ ví tự gọi lệnh chuyển tiền của Wallet, bỏ qua PIN và hạn mức	`gateway/src/app.ts` proxy toàn bộ `/api/wallets`; `wallet.routes.ts` dùng `requireOwnerOrAdmin`	Khoa (K9) + Đạt (Đ8b – chặn ở Gateway)
P0-2	Không có route confirm cho người dùng; xem/hủy được giao dịch của người khác	`payment.routes.ts`: confirm sau `requireInternalAuth`; getById, cancel không so chủ sở hữu	Khoa (K9)
P0-3	Checkout không gửi người nhận nên Hold luôn lỗi 400	`wallet.client.ts` createHold chỉ gửi amount, referenceId, expiresAt	Khoa (K9)
P0-4	Nhánh "bù trừ" tạo tiền từ không khi lỗi sau Capture	`payment.service.ts`: `walletClient.credit( ..., 'compensate_...')`	Khoa (K9)
P0-5	Hạn mức không biết người dùng ở cấp KYC nào (luôn = 0)	`authenticate.ts`: `decoded.kycTier ?? 0`; user-service chưa có publisher	Hữu (H7) + Đạt (Đ12) + Khoa (K8)
P0-6	Thông báo ≤ 3 giây không đạt vì Outbox quét 5 giây/lần	`outbox.scheduler.ts`, `outbox.job.ts` cron `*/5 * * * * *`	Nguyên (N8)
P0-7	transactionToken chưa có	`finvault-internal-`	Đạt (Đ8)

Mã	Vấn đề (nói đơn giản)	Bằng chứng trong code	Ai sửa, trong việc nào
	khóa ký riêng; Auth chưa dùng được thư viện chung	`auth` cố định issuer gateway; Auth build từ thư mục con	

1.5 Tải công việc

Thành viên	Vai trò chính	Must	Should	Could	Tổng
Đạt	Xác thực, PIN, transactionToken, Gateway, hạ tầng compose	5,0	0,5	1,5	7,0
Khoa	Ví, hạn mức, thanh toán P2P (đường găng)	5,0	0	0	5,0
Hữu	KYC, người dùng, merchant, kiểm thử nghiệm thu	4,75	1,75	0	6,5
Nguyên	Sự kiện, realtime, giao diện Web	5,0	0,5	1,0	6,5
Cộng		19,75	2,75	2,5	25,0

Quỹ 20 ngày công chỉ dư **0,25 ngày** sau phần Must → gần như không có chỗ trễ. Ai xong sớm thì hỗ trợ Khoa (đường găng). Thứ tự cắt khi trễ ở mục 12.

2. Bảng phân công tổng hợp

Toàn bộ việc của Tuần 2 trên một trang. Chi tiết từng việc (các bước làm, file cần sửa, test) nằm ở chương của từng người.

Mã	Việc	Người	Mức	Công	Ngày
Đ7	PIN giao dịch (đặt, đổi, xác minh, khóa 5 lần)	Đạt	Must	1,5	1–2
Đ8	transactionToken + nối dây Auth với thư viện chung	Đạt	Must	1,5	1–2
Đ8b	Middleware requireTransactionToken, bảng jti, chặn route tiền Wallet ở Gateway	Đạt	Must	0,5	3
Đ10	Modal PIN trên Web	Đạt	Must	1,0	4
Đ12 (K12)	Consumer của Wallet: tạo ví, cập nhật kycTier, ví merchant	Đạt (Khoa duyệt)	Must	0,5	3–4
B4	Refresh token qua body cho Mobile	Đạt	Should	0,5	nếu dư
Đ9	Đăng ký qua Outbox (thay gọi HTTP)	Đạt	Could	1,0	nếu dư
Đ11	Khởi tạo QR login	Đạt	Could	0,5	nếu dư
K7	Ví hệ thống SYSTEM (systemCode UNIQUE, seed migration)	Khoa	Must	1,0	1
K8	Limit Engine (hạn mức theo cấp KYC)	Khoa	Must	2,0	1–2
K9	Quote, checkout, confirm P2P; kiểm chủ sở hữu; bỏ nhánh bù; đóng route Wallet	Khoa	Must	1,5	3–4
B3 (Khoa)	Bỏ INTERNAL_KEY mặc định cứng ở Wallet/Payment	Khoa	Must	0,25	1
B5	Gom test song song thành `npm run test:concurrency`	Khoa	Must	0,25	2
H7	Publisher Outbox user-service + duyệt KYC đổi tier	Hữu	Must	1,25	1–2
H8	Hồ sơ CCCD hai mặt, URL ảnh cho Admin	Hữu	Must	1,0	2–3
H11	Tra cứu người nhận theo email/SĐT	Hữu	Must	0,5	3

Mã	Việc	Người	Mức	Công	Ngày
T1	Tự động hóa nghiệm thu (A3, A4, A9, R1, K1, S1–S16)	Hữu	Must	0,5	4–5
B1	Auth chuyển sang finvault-internal-auth, xóa requireGateway.ts cũ	Hữu	Must	0,5	1
B2	Bí mật & DB: bỏ mật khẩu cứng, chuyển secret sang _FILE	Hữu	Must	0,75	1–2
B3 (Hữu)	Bỏ INTERNAL_KEY mặc định cứng ở Admin/Auth	Hữu	Must	0,25	1
H9	Outlet, QR, ApiKey cho merchant	Hữu	Should	1,25	4
H10 rút gọn	Cổng merchant tối thiểu (xem outlet, QR)	Hữu	Should	0,5	5
N11	Thư viện finvault-events (phong bì, publish, consume + Inbox)	Nguyên	Must	0,75	1
N8	Realtime thông báo ≤ 3 giây	Nguyên	Must	1,25	1–3
K10	Màn chuyển tiền P2P trên Web	Nguyên	Must	1,0	3–4
N9	Màn Admin duyệt KYC	Nguyên	Must	1,5	4
B7	Sửa build FE + README gốc, README Wallet	Nguyên	Must	0,5	1–2
N10	Chuông thông báo	Nguyên	Should	0,5	5
N7	Quản lý thiết bị UserDevice	Nguyên	Could	1,0	nếu dư

3. Lịch 5 ngày và điểm bàn giao

Lịch được xếp để **mỗi người luôn có đầu vào sẵn** khi bắt tay. Không cần họp ngày 1: các quyết định M8–M25 đã chốt (mục 10), chỉ họp 15 phút nếu có bình luận phản đối trước 12:00 ngày 1. Mỗi ngày standup 10 phút: hôm qua xong gì, hôm nay làm gì, đang chặn ai / bị ai chặn.

Ngày	Đạt	Khoa	Hữu	Nguyên
1	Sửa Dockerfile + compose cho Auth (chặn cả nhóm); stub transactionToken; bắt đầu Đ7	K7 ví SYSTEM; khung LimitPolicy; B3; **công bố hợp đồng quote, checkout, LIMIT_EXCEEDED**	B1, B3; hợp đồng sự kiện cùng Nguyên; H7 phần publisher	**N11 finvault-events (ưu tiên số 1)**; bắt đầu N8; B7
2	Đ7 xong; Đ8 token thật	K8 Limit Engine + test song song; B5	H7 xong, **phát thật user.kyc_updated**; B2; bắt đầu H8	N8: làm giàu thông điệp, drain sau commit, ticket
3	Đ8b middleware, bảng jti, chặn Wallet ở Gateway; Đ12 consumer Wallet	K9 checkout, confirm, ownership, bỏ nhánh bù	H8 xong, route tier, H11; **bàn giao route Admin + URL ảnh**	N8 xong; bắt đầu K10 trên dữ liệu giả
4	Đ10 Modal PIN, ghép luồng P2P; Đ12 xong	K9 hoàn tất, **API P2P ổn định (không đổi tên trường)**	H9 outlet, QR, ApiKey (Should); bắt đầu T1	N9 Admin duyệt KYC; ghép K10 với API thật
5 (đóng băng)	Test bảo mật S1, S2, S5; đọc lại diff Gateway	Chạy toàn bộ nghiệm thu, sửa lỗi	Chạy T1 toàn hệ thống; H10 rút gọn nếu còn công	Đo độ trễ R1, diễn tập demo; N10 nếu còn công

Ngày 5 là ngày đóng băng: chỉ sửa lỗi, không thêm tính năng. Cuối ngày 4 mỗi người ghi lại việc chưa xong để quyết định cắt theo mục 12.

3.1 Điểm bàn giao bắt buộc

Hạn	Từ → Cho	Nội dung bàn giao	Nếu trễ thì
Cuối ngày 1	Nguyên → Cả nhóm	Package finvault-events chạy được ví dụ phát và nhận	Khoa, Hữu tạm ghi Outbox, chưa phát
Cuối ngày 1	Đạt → Khoa, Nguyên	Auth build được trong compose + stub transactionToken	Khoa, Nguyên thử bằng token giả trong test
Cuối ngày 1	Khoa → Nguyên	Hợp đồng quote, checkout, payload LIMIT_EXCEEDED	Nguyên dựng UI trên dữ liệu giả
Cuối ngày 2	Đạt → Khoa	verify PIN và token thật	Tiếp tục dùng stub, không chặn K9
Cuối ngày 2	Hữu → Khoa, Nguyên, Đạt	Sự kiện user.kyc_updated đã phát thật	Khoa điền tay cột kycTier để test
Giữa ngày 3	Khoa → Nguyên	Wallet có tier, payload lỗi hạn mức thật	Web hiển thị lỗi chung
Cuối ngày 3	Hữu → Nguyên	Route Admin tier và URL ảnh CCCD	N9 dời sang ngày 4, cắt N10
Giữa ngày 4	Khoa → Nguyên, Đạt	API P2P ổn định, không đổi tên trường	Ghép UI dời sang ngày 5, demo còn 1 luồng Web

3.2 Ai chờ ai (sơ đồ phụ thuộc bằng chữ)

- **N11 (Nguyên)** → Đ12 consumer Wallet (Đạt), H7 publisher (Hữu), N8 (Nguyên).
- **Đạt sửa build Auth (ngày 1)** → Đ7 PIN → Đ8 token → Đ8b middleware → K9 confirm (Khoa) → K10 + Đ10 (ghép UI).

- **H7 (Hữu)** → Đ12 cập nhật Wallet.kycTier → K8 tính hạn mức đúng cấp → kịch bản K1, A9.
- **K7 → K8 → K9 (Khoa)** là đường găng: Nguyên (K10) và Đạt (Đ10) đều chờ API của Khoa.
- **H8 + route Admin (Hữu)** → N9 màn duyệt KYC (Nguyên).

4. Nhiệm vụ của Đạt: xác thực, PIN và transactionToken

Mục	Nội dung
Phạm vi sở hữu	auth-service; packages/finvault-internal-auth; gateway; docker-compose.yml, init-secrets, biến môi trường; Modal PIN trên Web; middleware `requireTransactionToken` và bảng jti trong payment-service.
Tải công	Must 5,0 (Đ7 1,5 · Đ8 1,5 · Đ8b 0,5 · Đ10 1,0 · Đ12 0,5) · Should 0,5 (B4) · Could 1,5 (Đ9, Đ11)
Cần hiểu trước khi code	PIN giao dịch, bcrypt, cập nhật nguyên tử, JWT, ES256 vs HS256, Gateway Token vs transactionToken, jti, Docker secrets, Outbox/Inbox (cho Đ12). Xem mục 9.
Nhận đầu vào từ	Nguyên: finvault-events (cuối ngày 1, cho Đ12) · Khoa: route confirm (ngày 3) · Hữu: B1 xong trước khi Đạt sửa Auth.
Bàn giao cho	Khoa, Nguyên: Auth build được + stub token (cuối ngày 1); verify PIN và token thật (cuối ngày 2); Modal PIN (ngày 4).

4.1 Chi tiết từng việc

Đ8 (làm phần nền trước, ngày 1) · Nối dây Auth với thư viện chung + transactionToken	
Mục tiêu	Auth ký được transactionToken bằng khóa riêng; Payment kiểm được token đó; Gateway Token và transactionToken không dùng thay nhau được.
Hiện trạng	Auth build từ `./services/auth-service` nên không import được `packages/finvault-internal-auth`, không mount `gateway_public_key`. Thư viện cố định `issuer: finvault-gateway`, `audience: finvault-internal-services`. `init-secrets.js` chỉ sinh một cặp khóa (Gateway).
Các bước làm	**1. Ngày 1, việc đầu tiên:** đổi build context của Auth về gốc repo (giống cách đã làm cho wallet trong README mục 3), sửa `services/auth-service/Dockerfile` copy thêm thư viện; mount secret `gateway_public_key`. **2.** Tạo **stub** transactionToken (token cố định cho môi trường test) để Khoa và Nguyên làm trước, không phải chờ PIN thật. **3.** Sửa `scripts/init-secrets.js` sinh thêm cặp khóa **auth_txn** (ES256). Khóa riêng chỉ mount cho Auth; khóa công khai mount cho Payment. **4.** Trong Auth: hàm `signTransactionToken({sub, paymentId, amount})` với claims theo M8: `iss=finvault-auth`, `aud=finvault-payment`, `purpose=p2p_confirm`, `jti` (UUID), `iat`, `exp ≤ 60 giây`. `amount` là **chuỗi** VND. **5.** Trong `packages/finvault-internal-auth/src/index.ts`: thêm `verifyTransactionToken(token, publicKey)` kiểm đúng iss/aud/purpose (để Gateway Token bị từ chối) và trả claims. Viết test trong `tests/auth.test.cjs`.

	6. Route `POST /api/auth/pin/verify` (ở Đ7) trả `{transactionToken, expiresAt}` khi PIN đúng.
File liên quan	`services/auth-service/Dockerfile`, `docker-compose.yml`, `scripts/init-secrets.js`, `services/auth-service/src/services/authService.ts`, `packages/finvault-internal-auth/src/index.ts`
Xong khi	Auth khởi động trong compose; verify PIN trả token; token hết hạn sau 60 giây; dùng Gateway Token thay transactionToken bị từ chối và ngược lại.
Công / Mức	1,5 ngày · Must

Đ7 · PIN giao dịch

Mục tiêu	Người dùng có PIN 6 số riêng biệt với mật khẩu; PIN đúng mới được cấp transactionToken.
Hiện trạng	Chưa có bảng hay route PIN. `app.ts` chỉ mount `/api/auth` với signup, signin, refresh, signout. Mọi route auth đều công khai → cần danh tính đã xác thực (Gateway Token) để biết PIN của ai.
Các bước làm	**1.** Migration Prisma (tên `YYYYMMDD_auth_add_pin`): thêm `pinHash`, `pinFailedCount` (mặc định 0), `pinLockedUntil` (nullable) vào bảng User của Auth. **2.** Các route PIN đi qua `requireGateway` của thư viện chung để lấy `req.auth.userId`. **3.** `POST /api/auth/pin` {pin, password}: kiểm mật khẩu, chặn PIN yếu (000000, 123456, 111111, dãy tăng/giảm, 6 số giống nhau…) → 400 `PIN_WEAK`; lưu bcrypt cost 12 → 204. **4.** `PUT /api/auth/pin` {oldPin, newPin, password}: đổi PIN, áp dụng cùng luật khóa. **5.** `POST /api/auth/pin/verify` {pin, paymentId, amount}: chưa có PIN → 409 `PIN_NOT_SET`; đang khóa → 423 `PIN_LOCKED` kèm `lockedUntil` (kể cả nhập đúng); sai lần 1–4 → 401 `PIN_INVALID` kèm `attemptsLeft`; sai lần 5 → 401 `PIN_INVALID` kèm `lockedUntil` (khóa 30 phút); đúng → đặt lại bộ đếm, trả transactionToken. **6. Tăng bộ đếm nguyên tử** bằng một câu `UPDATE ... SET pinFailedCount = pinFailedCount + 1 ... RETURNING` (không đọc rồi ghi), để 2 request song song không vượt được giới hạn 5 lần. **7.** Không log PIN thô, không đưa PIN vào message lỗi.
Test phải viết	Sai 5 lần → khóa; lần 6 (PIN đúng) → 423; 2 request song song không vượt 5 lần; sau 30 phút (đồng hồ giả) mở khóa; grep log không có PIN; PIN yếu → 400.
Xong khi	Toàn bộ test trên xanh; kịch bản A3 chạy được.
Công / Mức	1,5 ngày · Must

Đ8b · Middleware requireTransactionToken, bảng jti, chặn Wallet ở Gateway

Mục tiêu	Confirm chỉ chạy khi có transactionToken hợp lệ, đúng giao dịch, dùng lần đầu; người dùng không gọi trực tiếp được route tiền của Wallet.
Các bước làm	**1.** Trong payment-service (Đạt sở hữu middleware và bảng này, Khoa sở hữu phần còn lại): bảng `UsedTransactionToken(jti UNIQUE,`

	`paymentId, usedAt)`. **2.** Middleware `requireTransactionToken`: đọc header `X-Transaction-Token`, gọi `verifyTransactionToken`; so `sub` = `req.auth.userId`, `paymentId` = `:id`, `amount` = số tiền của payment. Lỗi: 401 `TXN_TOKEN_INVALID` / `TXN_TOKEN_EXPIRED`, 403 `TXN_TOKEN_MISMATCH`. **3.** Ghi `jti` **trong cùng transaction** với việc chuyển payment sang PROCESSING (phối hợp với Khoa). Vi phạm UNIQUE → 409 `TXN_TOKEN_REUSED`. **4.** Gateway (`gateway/src/app.ts`): trả 403 khi người dùng gọi hold, capture, release, transfer, credit, debit của `/api/wallets`; chỉ cho `GET balance` và `GET history` (M23).
Test phải viết	S1, S2 (gọi route tiền Wallet bằng token USER → 403), S5 (đổi số tiền → 403), token dùng lần 2 → 409.
Xong khi	Test xanh; kịch bản A4 chạy được.
Công / Mức	0,5 ngày · Must

Đ12 (nhận từ K12 của Khoa) · Consumer sự kiện cho Wallet	
Mục tiêu	Wallet tự biết cấp KYC của người dùng và tự tạo ví khi có sự kiện, không phụ thuộc claim trong token.
Các bước làm	**1.** Dùng `finvault-events` của Nguyên (có Inbox chống trùng, retry, DLQ). Làm **sau khi Khoa xong K7**; Khoa duyệt PR (Khoa sở hữu `schema.prisma` của Wallet – nếu cần cột mới thì nhờ Khoa thêm). **2.** `user.registered` → tạo ví USER (nếu chưa có). **3.** `user.kyc_updated` → ghi `Wallet.kycTier` = `data.kycTier`. **4.** `merchant.status_updated` (APPROVED) → tạo ví `ownerType=MERCHANT`, `userId = merchant.id` (M22).
Test phải viết	S10: phát user.kyc_updated 2 lần cùng eventId → kycTier chỉ cập nhật 1 lần.
Xong khi	Phát user.kyc_updated thì Wallet.kycTier đổi trong vài giây; nhận trùng không đổi thêm.
Công / Mức	0,5 ngày · Must

Đ10 · Modal PIN trên Web	
Mục tiêu	Component dùng chung cho P2P (và thanh toán merchant ở Tuần 3).
Các bước làm	**1.** Ô nhập 6 số (che ký tự), nút xác nhận gọi `POST /api/auth/pin/verify`. **2.** Hiển thị `attemptsLeft` khi sai; khi `PIN_LOCKED` hiển thị đồng hồ đếm ngược tới `lockedUntil`. **3.** `PIN_NOT_SET` → chuyển sang màn đặt PIN (nhập mật khẩu + PIN 2 lần). **4.** Trả transactionToken cho màn P2P của Nguyên qua callback; **xóa PIN khỏi state ngay sau khi gửi**.
Xong khi	Demo: nhập sai 5 lần thấy thông báo khóa và thời gian còn lại; nhập đúng thì luồng P2P đi tiếp.
Công / Mức	1,0 ngày · Must

Mã	Việc (Should/Could)	Xong khi	Công
B4 (Should)	Header `x-client-platform: mobile` → refresh token trả trong body; Web giữ cookie	2 test: Mobile nhận trong body, Web nhận cookie, refresh chạy cả hai	0,5
Đ9 (Could)	signUp ghi `user.registered` vào Outbox thay cho gọi HTTP bằng `Promise.allSettled`. **Chỉ làm khi Đ12 đã chạy ổn**, có cờ cấu hình quay về HTTP	Tắt Wallet khi đăng ký, bật lại thì ví vẫn được tạo	1,0
Đ11 (Could)	QR login: bảng nonce Postgres có `expiresAt` (M21), route tạo và xác nhận	Có thể bỏ – tài liệu v2 đặt ở Tuần 3	0,5

4.2 Lưu ý kỹ thuật cho Đạt

- Access token HS256 hiện chỉ mang `{userId, role}` nên claim `kycTier` ở Gateway **luôn bằng 0**. Đừng dùng claim này để tính hạn mức; Wallet tự giữ tier (M13).
- **Không dùng chung khóa ES256 của Gateway** cho transactionToken: nếu lộ, kẻ tấn công giả được cả hai loại token.
- Rate limiter của Gateway lưu trong RAM (mất khi restart, không chia sẻ giữa instance) → chống dò PIN **phải dựa vào bộ đếm trong DB**.
- Hữu làm B1 (xóa `requireGateway.ts` cũ của Auth, bỏ cờ `x-gateway-verified`, bỏ `DISABLE_GATEWAY_CHECK`) trước – Đạt pull về rồi mới sửa Auth để không xung đột.

Checklist trước khi mở PR

- ☐ Không có PIN/token trong log và message lỗi
- ☐ Migration đặt tên `YYYYMMDD_auth_<nội dung>`
- ☐ `.env.example` của Auth cập nhật biến mới
- ☐ Test Đ7/Đ8/Đ8b xanh
- ☐ Có 1 người review ngoài tác giả (Khoa cho Đ8b, Đ12)

5. Nhiệm vụ của Khoa: ví, hạn mức và thanh toán P2P

Mục	Nội dung
Phạm vi sở hữu	wallet-service, payment-service (trừ middleware requireTransactionToken và bảng jti của Đạt). **Chỉ Khoa được đổi `schema.prisma` của Wallet.**
Tải công	Must 5,0 (K7 1,0 · K8 2,0 · K9 1,5 · B3 0,25 · B5 0,25). Khoa là **đường găng**: Nguyên và Đạt đều chờ API của Khoa, nên làm đúng thứ tự.
Cần hiểu trước khi code	Hold/Capture/Release, sổ cái kép, ví SYSTEM, Limit Engine, cập nhật nguyên tử & khóa hàng (FOR UPDATE), idempotency, máy trạng thái Payment, lũy đẳng. Xem mục 9.
Nhận đầu vào từ	Đạt: stub token (ngày 1), token thật (ngày 2), middleware Đ8b (ngày 3) · Hữu/Đạt: Wallet.kycTier (giữa ngày 3) · Nguyên: finvault-events (ngày 1).
Bàn giao cho	Nguyên: hợp đồng quote/checkout/LIMIT_EXCEEDED (cuối ngày 1), payload lỗi thật (giữa ngày 3), API P2P ổn định (giữa ngày 4).

5.1 Chi tiết từng việc

K7 · Ví hệ thống (SYSTEM)	
Mục tiêu	Mỗi loại ví hệ thống có đúng một bản ghi, tạo bằng migration, không tạo lười lúc chạy.
Hiện trạng	`ensureSystemWallet()` chạy trong constructor `WalletService` mỗi lần `new`, không có UNIQUE → có thể sinh nhiều ví SYSTEM; `getWalletIds` dùng `findFirst`. `Wallet.userId` đang `@unique` nên một chủ không có đồng thời ví USER và MERCHANT.
Các bước làm	**1.** Thêm cột `systemCode` (MAIN, FEE, TOPUP_POOL) với ràng buộc UNIQUE; index cho `ownerType`. **2.** Seed 3 ví bằng migration; bỏ `ensureSystemWallet()` khỏi constructor. **3.** `getWalletIds` tra theo `systemCode`, định tuyến theo `transferType`. **4.** Số dư cũ (nếu lệch) chuyển bằng **bút toán ADJUSTMENT**, không UPDATE tay (sổ cái append-only, sửa tay sinh `BALANCE_DRIFT`). **5.** Ví merchant: `ownerType=MERCHANT`, `userId = merchant.id` (M22).
Test	S9: khởi động 2 instance Wallet cùng lúc vẫn chỉ 1 ví mỗi mã; `checkSystemConservation` = 0.
Công / Mức	1,0 ngày · Must

K8 · Limit Engine (hạn mức theo cấp KYC)	
Mục tiêu	Mọi khoản chi (P2P, thanh toán) bị chặn nếu vượt hạn mức theo lần / ngày / tháng của cấp KYC người gửi.
Hiện trạng	Chưa có bảng. `wallet.routes.ts` còn `TODO(kycTier)`. Hold chưa kiểm hạn mức.
Các bước làm	**1.** Bảng `LimitPolicy(kycTier, perTxn, perDay, perMonth)` seed theo M9: Cấp 0 = 200.000 / 500.000 /

	2.000.000; Cấp 1 = 10tr / 20tr / 100tr; Cấp 2 = 50tr / 100tr / 500tr. **2.** Bảng `UserDailyUsage(userId, day, spentDay, monthKey, spentMonth …)`; cột `kycTier` trên Wallet (Đạt cập nhật qua consumer Đ12; rỗng thì đọc bù từ user-service nội bộ – M13). **3.** Hàm `checkAndReserve(userId, amount)` chạy **trong cùng transaction với hold**: kiểm theo thứ tự lần → ngày → tháng; **upsert nguyên tử** usage; chỉ khóa bản ghi usage của người gửi. **4. Kiểm hạn mức trước kiểm số dư** (để A9 trả LIMIT_EXCEEDED chứ không phải INSUFFICIENT_FUNDS). **5.** Ngày tính theo **GMT+7** (reset 00:00 giờ Việt Nam). Không tính TOPUP và REFUND. **6.** Release, hết hạn, hủy hold → **hoàn lại usage**. **7.** Lỗi: 422 `LIMIT_EXCEEDED` kèm `{limitType: PER_TXN|PER_DAY|PER_MONTH, limit, used, remaining, kycTier}`.
Test	Đã chi 450.000đ, chuyển 100.000đ → từ chối, remaining = 50.000; S6: 10 request song song vượt hạn mức → đúng số được phép qua; S7: release/hết hạn hoàn usage; mốc 00:00 GMT+7.
Công / Mức	2,0 ngày · Must

K9 · Quote, checkout, confirm P2P + sửa P0-1…P0-4	
Mục tiêu	Luồng P2P chạy đúng một lần, đúng người, không có đường vòng.
Các bước làm	**1.** `POST /api/payments/p2p/quote` {destinationUserId, amount} → `{amount, fee: 0, expiresAt}` (60 giây). **2.** `POST /api/payments/p2p/checkout`: đọc **header** `Idempotency-Key` (bỏ khỏi body); `amount` nhận chuỗi số, tính bằng Decimal/BigInt; bỏ `callbackTopic` do client gửi (server tự gán theo type). **3.** Thêm `destinationUserId`, `destinationOwnerType` vào checkout schema, bảng Payment (migration) và `WalletClient.createHold`; kiểm người nhận tồn tại, khác người gửi, ví ACTIVE. Hold TTL **5 phút**. (P0-3 – sửa cả hai phía trong cùng PR.) **4.** `POST /api/payments/:id/confirm` cho người dùng: `requireGateway` + `requireTransactionToken` (Đạt). Bản nội bộ đổi sang `/internal/payments/:id/confirm`. (P0-2) **5.** Confirm: PENDING → PROCESSING → gọi capture. Thành công → SUCCESS. Capture lỗi → FAILED, release hold, hoàn usage, ghi sự kiện. **Xóa nhánh `walletClient.credit(... 'compensate_')`** (P0-4). Không chắc kết quả (timeout) → giữ PROCESSING và thử lại capture (capture theo referenceId là lũy đẳng). **6.** Ánh xạ `HOLD_EXPIRED`, `HOLD_NOT_CAPTURABLE` → EXPIRED (410 `PAYMENT_EXPIRED`). Job payment-expiry chạy 30 giây–1 phút/lần (hiện 5 phút); bổ sung phát `payment.expired`. **7.** Kiểm chủ sở hữu cho `GET /:id`, `cancel`; `GET /payments` lọc theo `req.auth.userId` (chỉ ADMIN được truyền userId).

	8. Đóng route tiền của Wallet: hold, capture, release, transfer, credit, debit → `requireInternalOrAdmin`; người dùng chỉ còn GET balance, GET history (P0-1, M23).
File liên quan	`payment-service/src/services/payment.service.ts`, `routes/payment.routes.ts`, `clients/wallet.client.ts`, `validations/payment.validation.ts`, `jobs/payment-expiry.job.ts`, `prisma/schema.prisma`; `wallet-service/src/routes/wallet.routes.ts`, `controllers/wallet.controller.ts`, `services/wallet-hold.service.ts`
Test	S3, S4, S8, S15; một lần chuyển chỉ ra một cặp bút toán; A gọi transfer Wallet bằng token USER → 403.
Công / Mức	1,5 ngày · Must

Mã	Việc bù Tuần 1	Xong khi	Công
B3 (phần Khoa)	Bỏ `process.env.INTERNAL_KEY || '<chuỗi cứng>'` trong `wallet.client.ts`, `notification.client.ts`, `outbox.scheduler.ts`; thiếu biến thì dừng khi khởi động (fail-closed)	Khởi động không có INTERNAL_KEY báo lỗi rõ ràng	0,25
B5	Gom `tests/concurrency.ts` thành `npm run test:concurrency` có mã thoát; thêm 2 request hold song song trên cùng ví	Một lệnh chạy xanh trong CI	0,25

5.2 Lưu ý kỹ thuật cho Khoa

- **Thứ tự khóa:** giữ quy tắc `SELECT … FOR UPDATE` theo id tăng dần (đã có trong `ledger.service.ts`) để tránh deadlock khi A và B chuyển chéo cho nhau.
- **Capture lỗi nghĩa là tiền chưa rời ví** → chỉ release, không bao giờ credit để "bù".
- Hold TTL hiện là 15 phút ở Payment trong khi job Wallet nhả hold mỗi 10 giây → thống nhất 5 phút cho P2P.
- K12 (consumer Wallet) đã chuyển cho Đạt (M25); Khoa chỉ duyệt PR và thêm cột nếu Đạt cần.
- Ai xong sớm sẽ hỗ trợ Khoa: Đạt nhận phần kiểm chủ sở hữu và test bảo mật nếu Khoa trễ.

Checklist trước khi mở PR

- ☐ Tổng số dư hệ thống = 0 sau test
- ☐ Không còn nhánh `compensate_`
- ☐ Test song song xanh (`npm run test:concurrency`)
- ☐ Hợp đồng API khớp mục 8 – không tự đổi tên trường
- ☐ Migration đặt tên `YYYYMMDD_wallet_…` / `YYYYMMDD_payment_…`

6. Nhiệm vụ của Hữu: KYC, người dùng, merchant và kiểm thử

Mục	Nội dung
Phạm vi sở hữu	user-service, admin-service (API), merchant-service, tests/acceptance. Hữu sở hữu hợp đồng `user.kyc_updated` – nguồn dữ liệu của hạn mức.
Tải công	Must 4,75 (H7 1,25 · H8 1,0 · H11 0,5 · T1 0,5 · B1 0,5 · B2 0,75 · B3 0,25) · Should 1,75 (H9 1,25 · H10 rút gọn 0,5)
Cần hiểu trước khi code	KYC status vs tier, Outbox/Inbox, phong bì sự kiện, presigned URL & MinIO, rate limit, che dữ liệu nhạy cảm, AdminAuditLog. Xem mục 9.
Nhận đầu vào từ	Nguyên: finvault-events (cuối ngày 1).
Bàn giao cho	Khoa, Đạt, Nguyên: `user.kyc_updated` phát thật (cuối ngày 2) · Nguyên: route Admin tier + URL ảnh (cuối ngày 3), API tra cứu (giữa ngày 3).

6.1 Chi tiết từng việc

B1 + B3 (ngày 1, làm trước) · Dọn Auth và secret cứng	
Các bước làm	**1.** B1: Auth dùng `finvault-internal-auth`; xóa `services/auth-service/src/middlewares/requireGateway.ts` cũ, bỏ cờ `x-gateway-verified` và đường `DISABLE_GATEWAY_CHECK`. Lệnh Auth nội bộ giữ kiểm INTERNAL_KEY. (Đã làm giống vậy cho User và Merchant nên nhanh.) **Xong sớm để Đạt sửa Auth trên nền mới.** **2.** B3: bỏ INTERNAL_KEY mặc định cứng trong `config/env.ts` của Admin và Auth; thiếu biến thì dừng khi khởi động.
Xong khi	Gửi `x-gateway-verified: true` thẳng vào Auth → 401; khởi động thiếu INTERNAL_KEY → báo lỗi rõ.
Công / Mức	0,5 + 0,25 ngày · Must

H7 · Publisher Outbox user-service + duyệt KYC đổi tier	
Mục tiêu	Khi tier thay đổi, Wallet và Notification được báo qua sự kiện `user.kyc_updated`.
Hiện trạng	`updateKycTier` đã ghi `user.kyc_updated` vào Outbox trong cùng transaction (có test). Nhưng user-service **chỉ có worker nhận** (`events/worker.ts` chạy consumer), **chưa có publisher**; chưa có route HTTP; `setKycStatus(APPROVED)` không đổi tier (test còn khẳng định tier = 0).
Các bước làm	**1.** Viết publisher worker theo mẫu `merchant-service/src/events/outbox.ts` + `worker.ts` (claimEvent, publishOne, publishConfirmed) hoặc dùng finvault-events; phát lên `finvault.events`, routing key = eventType. **2.** `setKycStatus(APPROVED)` → tier 1 **trong cùng transaction** với ghi Outbox (M12). REJECTED sau APPROVED → tier 0. Sửa lại test cũ. **3.** Route `PATCH /api/admin/kyc/:userId/tier` {tier: 1|2, reason} cho nút "Nâng cấp 2 (mô phỏng liên kết

	ngân hàng)". Mọi lần đổi tier ghi `AdminAuditLog`. **4.** Payload v1: `data = {authUserId, kycTier, previousTier}` trong phong bì M10. Kiểm không phát trùng các bản ghi Outbox cũ.
Test	S11: duyệt rồi từ chối → tier về 0, có audit. Duyệt → Outbox có user.kyc_updated, Wallet nhận được.
Công / Mức	1,25 ngày · Must · **chặn Khoa và Nguyên → xong cuối ngày 2**

B2 · Bí mật và DB	
Các bước làm	**1.** Sinh mật khẩu cho `init-db.sql` từ `scripts/init-secrets.js` (bỏ mật khẩu cứng). **2.** Chuyển `ACCESS_TOKEN_SECRET` và `INTERNAL_KEY` sang biến `_FILE` (đọc từ Docker secret). **3.** Mount `gateway_public_key` cho Auth (phối hợp với Đạt). Role DB riêng từng service → hoãn Tuần 3 (ghi nợ).
Xong khi	Compose lên sạch; `git grep` không còn mật khẩu hay khóa cứng.
Công / Mức	0,75 ngày · Must

H8 · Hồ sơ CCCD hai mặt	
Hiện trạng	Đã có: giới hạn 5 MiB, JPEG/PNG, `sharp` chuẩn hóa, bucket private, presigned URL, worker dọn ảnh. Thiếu: chỉ 1 ảnh (`kycObjectKey`), URL sống 60 giây.
Các bước làm	**1.** Thêm `kycBackObjectKey` và số CCCD (log chỉ giữ 4 số cuối); bắt buộc đủ 2 mặt mới chuyển PENDING. Tái sử dụng cùng đường xử lý ảnh. **2.** `KYC_URL_TTL_SECONDS` mặc định 300 (test đặt 2). **3.** Route `GET /api/admin/kyc/:userId/documents` → `{frontUrl, backUrl, expiresAt}`, ghi `AdminAuditLog` mỗi lần xem. Danh sách KYC trả thêm cột `kycTier`.
Test	S12: người dùng thường xin URL của người khác → 403; URL hết hạn đúng hạn.
Công / Mức	1,0 ngày · Must

H11 · Tra cứu người nhận	
Các bước làm	**1.** `GET /api/users/lookup?q=` khớp **chính xác** email hoặc SĐT, trả `{authUserId, maskedName}`. **2.** Giới hạn 10 lần/phút/người dùng → 429. Không phân biệt "không tồn tại" với "không được phép" (cùng một dạng lỗi 404) để chống dò tài khoản (M18).
Test	S13: lần 11 trong 1 phút → 429; sai và đúng-nhưng-không-được-phép trả cùng dạng.
Công / Mức	0,5 ngày · Must · bàn giao giữa ngày 3 cho Nguyên

T1 (+ B6) · Tự động hóa nghiệm thu	
Các bước làm	**1.** Thêm vào `tests/acceptance` kịch bản A3, A4, A9, R1, K1 và S1–S16 (mục 11), gồm bước seed dữ liệu (`seed.mjs`). **2.** Chạy lại 8 kiểm tra cuối Tuần 1 trong cùng bộ (B6).

Xong khi	Một lệnh chạy toàn bộ và báo xanh trên compose sạch.
Công / Mức	0,5 ngày · Must

Mã	Việc (Should)	Xong khi	Công
H9	Bảng Outlet, QrCode, ApiKey (chỉ lưu hash, khóa thật hiện **đúng một lần** lúc tạo). Route tạo/liệt kê cho chủ merchant, kiểm quyền sở hữu. Khi duyệt merchant phát `merchant.status_updated` để Wallet tạo ví	Truy vấn lại không thấy khóa gốc; merchant A không thấy outlet của B	1,25
H10 rút gọn	Màn xem outlet và QR; bỏ biểu đồ, báo cáo (sang Tuần 3)	Merchant đăng nhập thấy outlet và QR của mình	0,5

6.2 Lưu ý kỹ thuật cho Hữu

- **Tier và status độc lập** trong code hiện tại → sửa trong cùng transaction với việc ghi Outbox, không tách hai bước.
- admin-service hiện chỉ có `/kyc`, `/kyc/:userId/review`, `/merchants`, `/audit-logs`; viết route mới theo mẫu `KycService`.
- `phone` không độc nhất → tra cứu theo SĐT trùng nhiều người thì trả lỗi dạng chung, không lộ danh sách.

Checklist trước khi mở PR

- ☐ Không log số CCCD đầy đủ
- ☐ Sự kiện đúng phong bì M10, routing key = eventType
- ☐ Có AdminAuditLog cho mọi hành động Admin
- ☐ Test đơn vị + tích hợp xanh
- ☐ `.env.example` cập nhật `KYC_URL_TTL_SECONDS`

7. Nhiệm vụ của Nguyên: sự kiện, realtime và giao diện

Mục	Nội dung
Phạm vi sở hữu	notification-service, packages/finvault-events, Web (màn P2P, thông báo, Admin KYC), tài liệu README.
Tải công	Must 5,0 (N11 0,75 · N8 1,25 · K10 1,0 · N9 1,5 · B7 0,5) · Should 0,5 (N10) · Could 1,0 (N7). Nhận màn P2P (K10) từ Khoa để cân tải.
Cần hiểu trước khi code	RabbitMQ exchange/routing key, phong bì sự kiện, publisher confirm, Outbox/Inbox/DLQ, Socket.IO room, socket ticket, idempotency phía client. Xem mục 9.
Nhận đầu vào từ	Khoa: hợp đồng quote/checkout (ngày 1), API thật (ngày 4) · Đạt: Modal PIN (ngày 4) · Hữu: lookup (ngày 3), route Admin (cuối ngày 3).
Bàn giao cho	Cả nhóm: finvault-events chạy được ví dụ (**cuối ngày 1**).

7.1 Chi tiết từng việc

N11 (ưu tiên số 1, ngày 1) · Thư viện finvault-events	
Mục tiêu	Một cách phát/nhận sự kiện thống nhất cho mọi service (hiện có 3 quy ước Outbox khác nhau).
Các bước làm	**1.** Tạo `packages/finvault-events`: kiểu phong bì `{eventId, eventType, eventVersion, occurredAt, data}` (M10); bảng exchange/routing key. **2.** Hàm `publish` dùng **publisher confirm** (chỉ coi là đã gửi khi broker xác nhận). **3.** Hàm `consume` có **Inbox** (lưu eventId đã xử lý), retry, DLQ – tái dùng cách làm hiện có ở notification/user-service. **4.** Ví dụ chạy được: một service phát, một service nhận, nhận trùng không xử lý hai lần. Không ép viết lại sự kiện cũ trong tuần này.
Công / Mức	0,75 ngày · Must · xong trước cuối ngày 1

N8 · Thông báo realtime ≤ 3 giây	
Hiện trạng	Đã có Socket.IO, ticket 60 giây dùng một lần, room `user:<id>`, consumer 3 exchange có retry + DLQ + Inbox. Thiếu: nghe `finvault.events`; Outbox quét 5 giây; nội dung không có tên người gửi.
Các bước làm	**1.** Thêm binding `user.#`, `merchant.#` (và sự kiện payment mới) vào `finvault.events`; giữ 3 hàng đợi cũ đến Tuần 3. **2.** Sau commit ở Wallet/Payment gọi ngay hàm **xả (drain) Outbox**; cron giữ làm lưới an toàn (dự phòng: hạ chu kỳ xuống 1 giây). Phối hợp với Khoa vì code nằm trong service của Khoa. **3.** Làm giàu thông điệp: “Bạn nhận 50.000đ từ Nguyễn V.” – thêm `counterpartyName`, `note` (consumer tra user, cache ngắn). Người gửi và người nhận có thông điệp riêng. **4.** Ánh xạ sự kiện → `NotificationType` (PAYMENT_EXPIRED, KYC_TIER_CHANGED đã có

	trong enum). **5.** FE dùng `auth` dạng hàm để **xin ticket mới mỗi lần kết nối lại** (`GET /api/notifications/socket-ticket`). **6.** Viết test R1 đo độ trễ thực.
Test	R1 (≥ 9/10 lần ≤ 3 giây), S14 (dùng lại ticket → bị từ chối), S16 (tắt RabbitMQ rồi bật lại → thông báo đến sau).
Công / Mức	1,25 ngày · Must

K10 · Màn chuyển tiền P2P trên Web	
Các bước làm	**1.** Nhập email/SĐT → gọi lookup → hiện `maskedName`. **2.** Nhập số tiền, ghi chú → gọi quote → màn xác nhận (phí 0đ, đếm ngược 60 giây). **3.** Sinh `Idempotency-Key` (UUID) **mỗi lần mở màn**, gửi checkout; bấm đúp không tạo hai giao dịch. **4.** Mở Modal PIN của Đạt → nhận transactionToken → gọi confirm. **5.** `LIMIT_EXCEEDED` → hiển thị "còn được chuyển X đ hôm nay" + gợi ý nâng cấp KYC; hiển thị thành công/thất bại, cập nhật số dư. **6.** Ngày 3 dựng trên dữ liệu giả theo hợp đồng của Khoa; ngày 4 ghép API thật.
Công / Mức	1,0 ngày · Must

N9 · Màn Admin duyệt KYC	
Các bước làm	**1.** Danh sách chờ duyệt có cột tier; xem hai mặt CCCD qua URL tạm (route của Hữu). **2.** Nút Duyệt, Từ chối kèm lý do, Nâng cấp 2 (M12).
Xong khi	Duyệt xong thấy tier đổi và AdminAuditLog ghi lại (kịch bản K1).
Công / Mức	1,5 ngày · Must

Mã	Việc	Xong khi	Công
B7 (Must)	Sửa 3 khai báo thừa trong `AdminTopups.tsx` để build FE qua; sửa README gốc (ví dụ socket sai: đúng là `auth: {ticket}`, không phải `auth: {token: userAccessToken}`) và README Wallet cũ (mô tả NestJS, `/internal/wallets` – không còn đúng)	`npm run build` FE xanh; README đúng	0,5
N10 (Should)	Chuông: badge chưa đọc, dropdown, nút "Đọc tất cả" gọi `PATCH /api/notifications/me/read-all` (backend đã có)	Bấm Đọc tất cả, badge về 0 ngay	0,5
N7 (Could)	Bảng UserDevice lưu thiết bị đăng nhập	Chỉ làm khi dư công	1,0

7.2 Lưu ý kỹ thuật cho Nguyên

- Outbox ví hiện chỉ đẩy `{userId, type, message}` (không có phong bì); user và merchant đã dùng phong bì. Consumer phải đọc được cả hai trong Tuần 2.
- Repo FE nằm ngoài phạm vi đã rà soát → sau ngày 1, Nguyên và Đạt tự định giá lại các việc giao diện và báo nếu lệch quá 0,5 ngày.

Checklist trước khi mở PR

- ☐ Không hiển thị token/PIN trong console FE
- ☐ Idempotency-Key sinh mỗi lần mở màn
- ☐ Thông báo đến cả người gửi lẫn người nhận
- ☐ Đo R1 và ghi kết quả
- ☐ README đã sửa ví dụ socket

8. Hợp đồng API và sự kiện Tuần 2

Đây là "luật chung" giữa các người. **Ai muốn đổi tên trường, mã lỗi hay mã trạng thái phải báo chủ sở hữu trước.** Mọi route qua Gateway có tiền tố `/api`. Lỗi dùng chung dạng `{code, message, details?}`; `message` không được chứa PIN, token hay số CCCD đầy đủ.

8.1 API

Route	Chủ	Quyền	Yêu cầu	Trả về và lỗi
POST /auth/pin	Đạt	USER	{pin, password}	204; 400 PIN_WEAK; 401 sai mật khẩu
PUT /auth/pin	Đạt	USER	{oldPin, newPin, password}	204; 401 PIN_INVALID; 423 PIN_LOCKED
POST /auth/pin/verify	Đạt	USER	{pin, paymentId, amount}	{transactionToken, expiresAt}; 401 PIN_INVALID + attemptsLeft; 409 PIN_NOT_SET; 423 PIN_LOCKED + lockedUntil
GET /users/lookup?q=	Hữu	USER	email hoặc SĐT đầy đủ	{authUserId, maskedName}; 404 dạng chung; 429
POST /users/kyc/documents	Hữu	USER	front, back, idNumber (multipart)	202 PENDING; 400 thiếu mặt
PATCH /admin/kyc/:userId/tier	Hữu	ADMIN	{tier: 1|2, reason}	200; ghi AdminAuditLog
GET /admin/kyc/:userId/documents	Hữu	ADMIN	—	{frontUrl, backUrl, expiresAt}
POST /merchants/:id/outlets	Hữu	MERCHANT	{name, address}	201 outlet; 403 sai chủ
POST /merchants/:id/api-keys	Hữu	MERCHANT	{label}	201 kèm secret hiện một lần
POST /payments/p2p/quote	Khoa	USER	{destinationUserId, amount}	{amount, fee: 0, expiresAt}; 422 LIMIT_EXCEEDED
POST /payments/p2p/checkout	Khoa	USER	Header Idempotency-Key; {destinationUserId, amount, note?}	201 {paymentId, status: PENDING, holdExpiresAt}; 422 LIMIT_EXCEEDED; 422 INSUFFICIENT_FUNDS
POST /payments/:id/confirm	Khoa	USER (chủ)	Header X-Transaction-Token	200 {status: SUCCESS}; 401/403/409 TXN_TOKEN_*; 409 PAYMENT_NOT_PENDING; 410 PAYMENT_EXPIRED
POST /payments/:id/cancel	Khoa	USER (chủ)	—	200 CANCELLED; 403 sai chủ
GET /payments/:id, GET /payments	Khoa	USER (chủ)	—	403 nếu không phải chủ
GET /wallets/:userId/balance, /history	Khoa	USER	—	Chỉ hai route này dành cho người dùng (M23)
GET /notifications/socket-ticket	Nguyên	USER	—	{ticket, expiresAt}; mỗi lần gọi một ticket mới

8.2 Ví dụ payload LIMIT_EXCEEDED

```
422 {"code":"LIMIT_EXCEEDED","message":"Vượt hạn mức ngày","details":
{"limitType":"PER_DAY","limit":"500000","used":"450000","remaining":"50000",
"kycTier":0}}
```

8.3 Sự kiện trên exchange finvault.events

Phong bì chung (M10): `{eventId, eventType, eventVersion, occurredAt, data}`. Routing key = eventType. Exchange loại topic.

eventType	Phát từ	Nhận	data
user.registered	Auth (Outbox, Đ9 – Could)	User, Wallet	{authUserId, email, phone?}
user.kyc_updated	User (Hữu)	Wallet, Notification	{authUserId, kycTier, previousTier}
merchant.status_updated	Merchant (Hữu)	Wallet, Notification	{merchantId, status}
payment.succeeded	Payment (Khoa)	Notification	{paymentId, senderId, receiverId, amount}
payment.failed	Payment (Khoa)	Notification	{paymentId, senderId, reason}
payment.expired	Payment (Khoa)	Notification	{paymentId, senderId}
wallet.limit_exceeded	Wallet (Khoa), tùy chọn	Notification	{userId, limitType, kycTier}

Quy tắc phiên bản: thêm trường thì giữ `eventVersion`; đổi hoặc xóa trường thì tăng `eventVersion` và consumer hỗ trợ cả hai bản trong một tuần. `user.registered` đã có consumer ở user-service (`registration.ts`) → giữ đúng tên khi Auth chuyển sang Outbox.

9. Giải thích các khái niệm quan trọng

Mỗi khái niệm gồm: **là gì**, **vì sao cần**, và **trong FinVault ở đâu / ví dụ**. Đọc trước khi bắt tay vào việc của mình; nếu vẫn chưa rõ, hỏi chủ sở hữu ở bảng mục 13.

Nhóm 1 – Kiến trúc chung

Khái niệm	Là gì	Vì sao cần	Trong FinVault / ví dụ
Microservice	Hệ thống được chia thành nhiều service nhỏ, mỗi service một cơ sở dữ liệu riêng và một trách nhiệm riêng.	Một service lỗi không kéo sập cả hệ thống; mỗi người sở hữu một service, ít đụng nhau.	FinVault có 8 service: gateway, auth, user, merchant, wallet, payment, notification, admin.
API Gateway	"Cổng vào" duy nhất từ bên ngoài (cổng 3000). Mọi request của Web/Mobile đi qua đây, Gateway xác thực rồi chuyển tiếp vào service bên trong.	Chỉ một chỗ phải bảo vệ; service bên trong không mở cổng ra ngoài.	Gateway chặn người dùng gọi `POST /api/wallets/:id/transfer` (M23).
Đường găng (critical path)	Chuỗi việc dài nhất mà nếu một việc trễ thì cả tuần trễ theo.	Biết ai cần được ưu tiên hỗ trợ.	K7 → K8 → K9 của Khoa: Nguyên và Đạt đều chờ API P2P.
Must / Should / Could	Ba mức ưu tiên. Must bắt buộc, Should nên có, Could làm khi dư.	Khi trễ, biết cắt gì mà không hỏng DoD.	PIN là Must; QR login (Đ11) là Could.

Nhóm 2 – Xác thực và bảo mật

Khái niệm	Là gì	Vì sao cần	Trong FinVault / ví dụ
JWT (JSON Web Token)	Chuỗi gồm 3 phần header.payload.signature. Payload chứa "claims" (ví dụ sub = ai, exp = hết hạn lúc nào). Chữ ký chứng minh token không bị sửa.	Service kiểm token mà không cần hỏi lại DB.	Access token, Gateway Token và transactionToken đều là JWT.
HS256 và ES256	HS256 dùng **một khóa bí mật chung** để ký và kiểm. ES256 dùng **cặp khóa**: khóa riêng để ký, khóa công khai để kiểm.	Với ES256, service chỉ giữ khóa công khai → không thể giả token dù bị lộ.	Gateway Token và transactionToken dùng ES256; access token còn HS256 (nợ kỹ thuật).
Ba loại token	**Access token**: người dùng đã đăng nhập (sống 15 phút). **Gateway Token**: Gateway ký lại để service bên trong biết "ai đang gọi" (60 giây). **transactionToken**: Auth cấp sau khi PIN đúng, chứng minh "chủ ví đồng ý đúng số tiền này cho đúng giao dịch này" (60 giây, dùng 1 lần).	Mỗi loại trả lời một câu hỏi khác nhau; không được dùng thay nhau.	Khóa ký transactionToken (auth_txn) khác khóa Gateway; iss/aud/purpose khác nhau.
Claims iss, aud, sub, exp, jti	iss = ai phát hành; aud = dành cho ai; sub = người dùng; exp = hết hạn; jti = mã duy nhất của token.	Kiểm iss/aud để token loại này không dùng được ở chỗ khác; jti để chặn dùng lại.	transactionToken: iss=finvault-auth, aud=finvault-payment, purpose=p2p_confirm.
Token dùng một lần (jti + UNIQUE)	Khi dùng token, ghi jti vào bảng có ràng buộc UNIQUE. Lần thứ hai ghi trùng → DB từ chối.	Kẻ nghe lén lấy được token cũng không "phát lại" được.	Confirm lần 2 cùng token → 409 TXN_TOKEN_REUSED.
PIN giao dịch	Mã 6 số, tách biệt mật khẩu,	Mật khẩu bị lộ (hoặc máy	Sai 5 lần → khóa 30 phút;

Khái niệm	Là gì	Vì sao cần	Trong FinVault / ví dụ
	chỉ để xác nhận chuyển tiền.	đang đăng nhập bị người khác cầm) vẫn chưa chuyển được tiền.	đặt/đổi PIN phải nhập mật khẩu.
bcrypt	Hàm băm mật khẩu chậm có "muối" (salt). Cost 12 nghĩa là đủ chậm để dò thử rất tốn thời gian.	PIN chỉ có 1 triệu khả năng; băm nhanh (SHA-256) thì lộ DB là dò ra ngay.	Lưu `pinHash` bằng bcrypt cost 12, không bao giờ lưu PIN thô.
Fail-closed	Thiếu cấu hình bảo mật thì dừng hẳn, không chạy với giá trị mặc định.	Giá trị mặc định cứng như `default_internal_secret_key_123456` ai cũng biết.	B3: thiếu INTERNAL_KEY thì service báo lỗi khi khởi động.
Docker secrets / biến _FILE	Bí mật được mount thành file trong container thay vì truyền qua biến môi trường.	Biến môi trường dễ lộ qua log, `docker inspect`.	B2: `ACCESS_TOKEN_SECRET_FILE`, `INTERNAL_KEY_FILE`.
Rate limit	Giới hạn số lần gọi trong một khoảng thời gian.	Chống dò tài khoản, dò PIN.	Tra cứu người nhận 10 lần/phút → lần 11 trả 429.

Nhóm 3 – Tiền và cơ sở dữ liệu

Khái niệm	Là gì	Vì sao cần	Trong FinVault / ví dụ
Transaction (giao dịch DB)	Nhóm câu lệnh DB chạy "tất cả hoặc không gì cả".	Không bao giờ có trạng thái nửa vời (đã trừ tiền A mà chưa cộng B).	Hold + cộng hạn mức chạy trong cùng một transaction.
Race condition và cập nhật nguyên tử	Hai request đến cùng lúc cùng đọc giá trị cũ rồi cùng ghi → vượt giới hạn. Cập nhật nguyên tử = để DB tự tính trong một câu lệnh.	Đọc-rồi-ghi là lỗi kinh điển với tiền và bộ đếm.	`UPDATE ... SET pinFailedCount = pinFailedCount + 1 ... RETURNING`; upsert UserDailyUsage.
Khóa hàng FOR UPDATE và deadlock	`SELECT ... FOR UPDATE` khóa dòng để request khác phải chờ. Deadlock: A khóa X chờ Y, B khóa Y chờ X.	Khóa luôn theo **id tăng dần** thì không thể deadlock.	`ledger.service.ts` sắp id tăng dần rồi khóa hai ví.
Hold / Capture / Release	**Hold** = tạm giữ tiền (available giảm, held tăng), chưa ghi sổ cái. **Capture** = chuyển thật sang ví đích bằng bút toán kép. **Release** = trả tiền giữ về available.	Cho phép "giữ chỗ" trong lúc chờ PIN mà không chuyển tiền ngay.	Hold P2P sống 5 phút; scheduler 10 giây/lần nhả hold hết hạn.
Sổ cái kép (double-entry ledger)	Mỗi lần chuyển tạo 2 dòng bằng nhau: ghi nợ ví nguồn, ghi có ví đích. Sổ cái chỉ thêm, không sửa (trigger `prevent_ledger_mutation`).	Tổng mọi số dư luôn bằng 0 → dễ phát hiện sai lệch.	Sai thì sửa bằng bút toán đảo, không UPDATE.
Ví hệ thống (SYSTEM)	Ví của chính hệ thống, làm đối ứng cho nạp tiền, hoàn tiền, phí.	Tiền nạp vào ví người dùng phải "đến từ đâu đó" để tổng vẫn = 0.	K7: 3 ví MAIN, FEE, TOPUP_POOL; mỗi mã đúng một bản ghi.
Đối soát (reconciliation)	So số dư trong bảng Wallet với tổng sổ cái; lệch thì báo BALANCE_DRIFT.	Phát hiện lỗi tiền âm thầm.	`checkSystemConservation` phải bằng 0 sau mỗi lần chạy test.
Limit Engine (hạn mức)	Bộ kiểm "khoản chi này có được phép không" theo cấp KYC: theo lần, theo ngày, theo tháng.	Yêu cầu pháp lý và chống gian lận; tài khoản chưa định danh chỉ được chuyển ít.	Cấp 0 đã chi 450.000đ, chuyển thêm 100.000đ → 550.000 > 500.000 → từ chối, remaining = 50.000.
KYC status khác KYC tier	**status** (NONE, PENDING, APPROVED, REJECTED) = hồ sơ đang ở bước nào. **tier** (0, 1, 2) = được chuyển bao nhiêu.	Code hiện tại để chúng độc lập nên duyệt xong vẫn tier 0.	M12: APPROVED → tier 1 trong cùng transaction; tier 2 do Admin bấm nâng cấp.
Idempotency (lũy đẳng)	Gửi cùng một yêu cầu nhiều	Mạng chập chờn, người	Header `Idempotency-Key`

Khái niệm	Là gì	Vì sao cần	Trong FinVault / ví dụ
	lần cho cùng kết quả như gửi một lần.	dùng bấm đúp – không được trừ tiền hai lần.	(UUID do client sinh) + `@@unique([userId, idempotencyKey])`.
Máy trạng thái Payment	Danh sách trạng thái và các bước chuyển được phép.	Chặn chuyển trạng thái vô lý (ví dụ SUCCESS → PENDING).	PENDING → PROCESSING → SUCCESS | FAILED; PENDING → CANCELLED | EXPIRED (M14).
"Timeout khác thất bại"	Gọi sang service khác bị timeout không có nghĩa là thất bại – có thể bên kia đã làm xong.	Tưởng thất bại rồi "bù tiền" sẽ nhân đôi tiền (lỗi P0-4).	Không chắc kết quả → giữ PROCESSING, thử lại capture (lũy đẳng theo referenceId).
Migration (Prisma)	File SQL mô tả thay đổi cấu trúc DB, chạy theo thứ tự.	Mọi máy có cùng cấu trúc DB; tránh sửa tay.	Đặt tên `YYYYMMDD_<service>_<nội dung>`; chỉ chủ service sửa `schema.prisma`.

Nhóm 4 – Sự kiện và realtime

Khái niệm	Là gì	Vì sao cần	Trong FinVault / ví dụ
Message broker, RabbitMQ	Hệ thống trung gian nhận tin từ bên phát và giao cho bên nhận, kể cả khi bên nhận đang tạm tắt.	Service không phải gọi trực tiếp nhau; một bên chết không làm bên kia chết theo.	RabbitMQ chạy trong docker-compose.
Exchange, routing key, binding, queue	Bên phát gửi vào **exchange** kèm **routing key**. Mỗi **queue** đăng ký (binding) theo mẫu, ví dụ `user.#` nhận mọi sự kiện bắt đầu bằng `user.`.	Thêm bên nhận mới không phải sửa bên phát.	Exchange `finvault.events` (topic), routing key = eventType.
Phong bì sự kiện (envelope)	Khung chung bọc mọi sự kiện: eventId, eventType, eventVersion, occurredAt, data.	Mọi consumer đọc cùng một kiểu; dễ chống trùng và nâng phiên bản.	M10.
Outbox	Bảng sự kiện ghi **cùng transaction** với dữ liệu nghiệp vụ; publisher đọc bảng rồi đẩy sang RabbitMQ.	Không bao giờ có chuyện tiền đã đổi mà sự kiện mất (hoặc sự kiện có mà tiền chưa đổi).	Wallet, Payment, User, Merchant đều có Outbox.
Publisher confirm	RabbitMQ báo lại "đã nhận" cho bên phát. Chỉ khi đó mới đánh dấu Outbox đã gửi.	Tránh mất tin khi broker sập giữa chừng.	`publishConfirmed` trong `merchant-service/src/events/amqp.ts`.
Inbox	Ở phía nhận, lưu eventId đã xử lý; nhận trùng thì bỏ qua.	RabbitMQ chỉ đảm bảo giao **ít nhất một lần** → có thể trùng.	Test 20 lần giao đồng thời chỉ tạo 1 thông báo.
Retry và DLQ	Xử lý lỗi thì thử lại sau; quá số lần thì đưa vào Dead Letter Queue để người xem.	Một tin lỗi không chặn cả hàng đợi.	Consumer notification, user-service đã có.
Drain Outbox sau commit	Ngay sau khi commit, gọi hàm đẩy Outbox luôn thay vì chờ cron.	Cron 5 giây làm độ trễ trung bình 2,5 giây, tối đa 5 giây → không đạt ≤ 3 giây.	N8; cron giữ làm lưới an toàn.
Socket.IO, room	Kết nối hai chiều giữa server và trình duyệt. Room = nhóm socket; mỗi người dùng một room `user:<id>`.	Server chủ động đẩy thông báo, không cần Web hỏi liên tục.	Notification đẩy vào room của người gửi và người nhận.
Socket ticket	Vé dùng một lần, sống 60 giây, xin qua `GET /api/notifications/socket-ticket` rồi dùng để kết nối.	WebSocket không gửi header Authorization ổn định; vé ngắn hạn an toàn hơn đưa access token.	Mỗi lần kết nối lại phải xin vé mới (S14).
MinIO và presigned URL	MinIO là kho file kiểu S3.	Ảnh CCCD để private; chỉ	`KYC_URL_TTL_SECONDS` =

Khái niệm	Là gì	Vì sao cần	Trong FinVault / ví dụ
	Presigned URL là đường link có chữ ký, chỉ sống một thời gian.	người có quyền nhận link tạm.	300; Admin xin URL thì ghi AdminAuditLog.

10. Các quyết định đã chốt (M8–M25)

Đã chốt theo phương án tối ưu. Ai không đồng ý thì bình luận trực tiếp trước **12:00 ngày 1**; không ai phản đối coi như đã chốt.

Mã	Nội dung	Quyết định	Người quyết
M8	Định dạng transactionToken	ES256, khóa riêng auth_txn. Claims: iss=finvault-auth, aud=finvault-payment, sub, paymentId, amount (chuỗi), purpose=p2p_confirm, jti, iat, exp ≤ 60s. Header X-Transaction-Token. Lỗi 401 TXN_TOKEN_INVALID/EXPIRED, 409 REUSED, 403 MISMATCH	Đạt + Khoa
M9	Bảng LimitPolicy	Cấp 0: 200k / 500k / 2tr; Cấp 1: 10tr / 20tr / 100tr; Cấp 2: 50tr / 100tr / 500tr (lần/ngày/tháng). Reset 00:00 GMT+7. Tính khi Hold, hoàn khi Release/Expire/Cancel. Không tính TOPUP, REFUND	Khoa + Hữu
M10	Sự kiện dùng chung	Phong bì {eventId, eventType, eventVersion, occurredAt, data}; exchange finvault.events (topic); routing key = eventType	Nguyên + Hữu
M11	Luồng giao diện P2P	Nhập người nhận → quote → xác nhận → checkout → Modal PIN → confirm	Khoa + Đạt
M12	Duyệt KYC và tier	APPROVED → tier 1 cùng transaction; REJECTED sau APPROVED → tier 0; tier 2 do Admin nâng (mô phỏng)	Hữu + Nguyên
M13	Nguồn tier cho hạn mức	Wallet lưu kycTier riêng, cập nhật bằng consumer; rỗng thì đọc bù từ user-service. Claim trong token chỉ để hiển thị	Khoa
M14	Máy trạng thái Payment	PENDING → PROCESSING → SUCCESS | FAILED; PENDING → CANCELLED | EXPIRED. REFUNDING/REFUNDED để Tuần 3	Khoa
M15	Mốc thời gian	Hold P2P 5 phút; transactionToken 60s; quote 60s; khóa PIN 30 phút; socket ticket 60s; URL ảnh KYC 300s	Cả nhóm
M16	Phí P2P	0đ trong Tuần 2; ví FEE để Tuần 3	Khoa + Hữu
M17	Chính sách PIN	6 số; bcrypt 12; chặn dãy dễ đoán; sai 5 lần khóa 30 phút; đúng thì reset bộ đếm; đặt/đổi cần mật khẩu; quên PIN ngoài phạm vi	Đạt
M18	Tra cứu người nhận	Khớp chính xác email/SĐT; trả {authUserId,	Hữu

Mã	Nội dung	Quyết định	Người quyết
		maskedName}; 10 lần/phút; không phân biệt không tồn tại / không được phép	
M19	Hồ sơ KYC	2 ảnh + số CCCD; log chỉ 4 số cuối	Hữu
M20	Nền tảng demo	Web. Mobile chỉ đăng nhập nếu còn sức	Cả nhóm
M21	Nonce QR login	Bảng Postgres có expiresAt (không thêm Redis)	Đạt
M22	Khóa ví merchant	ownerType=MERCHANT, userId = merchant.id	Khoa + Hữu
M23	Quy tắc truy cập ví	Người dùng chỉ GET balance, GET history; hold/capture/release/transfer /credit/debit chỉ cho nội bộ hoặc Admin	Khoa + Đạt
M24	Hạ H9	H9 (outlet, QR, ApiKey) từ Must xuống Should	Cả nhóm
M25	Chuyển K12	Consumer Wallet chuyển từ Khoa sang Đạt; Khoa duyệt PR	Cả nhóm

11. Kịch bản nghiệm thu

11.1 Năm kịch bản gốc

Mã	Kịch bản	Chuẩn bị và bước làm	Kỳ vọng	Viết / chạy
A3	Khóa PIN	Tài khoản có PIN. Verify sai 5 lần, lần 6 dùng PIN đúng	Lần 5 trả lockedUntil; lần 6 trả 423 PIN_LOCKED dù đúng; sau 30 phút (đồng hồ giả) mở khóa	Đạt / Hữu
A4	Token dùng lại	Lấy token cho paymentId P. Confirm 2 lần; thêm 1 lần với token quá 65 giây	Lần 1 SUCCESS; lần 2 409 TXN_TOKEN_REUSED; quá hạn 401 TXN_TOKEN_EXPIRED; chỉ một cặp bút toán	Đạt / Hữu
A9	Hạn mức Cấp 0	Tài khoản chi@demo.vn (Cấp 0), nạp thêm 1.000.000đ. Chuyển 600.000đ	422 LIMIT_EXCEEDED, limitType PER_TXN; số dư và usage không đổi. Biến thể: 3 × 200.000 → lần 3 bị PER_DAY	Khoa / Hữu
R1	Thông báo realtime	A chuyển 50.000đ cho B; B mở Web, đã kết nối socket bằng ticket	Popup ở máy B trong ≤ 3 giây ở ít nhất 9/10 lần	Nguyên / Hữu ghi nhận
K1	Duyệt KYC nâng hạn mức	User nộp 2 mặt CCCD; Admin mở ảnh, bấm Duyệt; user chuyển 1.000.000đ	Tier lên 1 trong vài giây; chuyển thành công (tier 0 sẽ bị chặn)	Hữu / Nguyên trên UI

11.2 Kiểm thử mới S1–S16

Mã	Bài kiểm thử	Kỳ vọng	Người
S1	USER gọi thẳng POST /api/wallets/:userId/transfer	403	Đạt
S2	USER gọi hold, capture, release, credit, debit trực tiếp	403 mỗi route	Đạt
S3	A đọc GET /payments/:id của B, rồi cancel	403 cả hai	Khoa
S4	Confirm paymentId người khác bằng token của mình	403 TXN_TOKEN_MISMATCH	Khoa
S5	Đổi số tiền sau khi lấy token	403 TXN_TOKEN_MISMATCH	Đạt
S6	10 checkout song song vượt hạn mức ngày	Đúng số request được qua; usage không vượt trần	Khoa
S7	Release và hết hạn hold hoàn usage	Usage về đúng giá trị trước hold	Khoa
S8	Capture lỗi (tắt Wallet giữa chừng)	Payment FAILED, hold release, người gửi không được cộng thêm tiền	Khoa
S9	Hai instance Wallet khởi động cùng lúc	Mỗi systemCode đúng 1 ví; tổng số dư = 0	Khoa
S10	Phát user.kyc_updated 2 lần	Wallet.kycTier cập nhật 1	Đạt (Đ12)

Mã	Bài kiểm thử	Kỳ vọng	Người
	cùng eventId	lần	
S11	Duyệt KYC rồi từ chối lại	Tier về 0, có AdminAuditLog	Hữu
S12	USER xin URL ảnh KYC của người khác	403; URL hết hạn sau 300 giây	Hữu
S13	Tra cứu người nhận 11 lần/phút	Lần 11 trả 429; lỗi cùng dạng	Hữu
S14	Dùng lại socket ticket đã dùng	Kết nối bị từ chối	Nguyên
S15	Idempotency-Key trùng 2 lần liên tiếp	Cùng paymentId, một hold	Khoa
S16	Tắt RabbitMQ khi chuyển tiền rồi bật lại	Giao dịch vẫn thành công; thông báo đến sau	Nguyên

11.3 Tài khoản seed

Tài khoản	Vai trò	KYC	Số dư / ghi chú
an@demo.vn	Người dùng	Cấp 2	1.250.000đ – người chuyển chính
binh@demo.vn	Người dùng	Cấp 1	300.000đ – người nhận
chi@demo.vn	Người dùng	Cấp 0	100.000đ – test hạn mức A9 (nạp thêm 1.000.000đ trước khi chạy)
highlands@demo.vn	Merchant	–	0đ – điểm bán có QR
admin@demo.vn	ADMIN	–	Duyệt KYC

12. Rủi ro, thứ tự cắt và nợ kỹ thuật

Rủi ro	Mức	Cách giảm	Theo dõi
Khoa trễ K8/K9 (tải cao nhất)	Cao	Bàn giao hợp đồng ngày 1; Nguyên dựng UI trên dữ liệu giả; Đạt hỗ trợ kiểm chủ sở hữu và test bảo mật	Khoa
Auth không build được với thư viện chung	TB	Làm ngay ngày 1; bế tắc thì tạm sao chép hàm verify vào Auth	Đạt
Hai người sửa cùng schema.prisma	TB	Chỉ chủ sở hữu đổi schema (mục 13)	Mỗi chủ service
Độ trễ realtime > 3 giây	Cao nếu không sửa	Drain Outbox sau commit; dự phòng hạ chu kỳ xuống 1 giây	Nguyên
Tier không đến Wallet	Cao nếu bỏ H7	H7 xong ngày 2; Wallet đọc bù từ user-service (M13)	Hữu, Khoa
Demo chạy nhưng lỗ hổng còn (gọi thẳng API)	TB	S1, S2, S3 bắt buộc trước khi đóng băng	Đạt
Lỗi nhất quán tiền	Thấp, nghiêm trọng	checkSystemConservation sau mỗi lần chạy tự động	Khoa
Ước lượng giao diện chưa kiểm chứng (repo FE riêng)	Chắc chắn	Nguyên, Đạt tự định giá lại sau ngày 1	Nguyên, Đạt

12.1 Thứ tự cắt khi trễ (cắt từ trên xuống, dừng khi kịp)

1. Đ11 QR login và N7 UserDevice
2. Đ9 đăng ký qua Outbox (vẫn chạy HTTP cũ)

3. N10 chuông thông báo
4. B4 refresh token Mobile
5. H10 rút gọn (cổng merchant)
6. H9 outlet, QR, ApiKey
7. Cuối cùng: demo chỉ còn luồng P2P có PIN và hạn mức, KYC ở dạng chụp màn hình

Không bao giờ cắt phần lõi: PIN, transactionToken, hạn mức, P2P, chặn route Wallet, publisher tier, xóa secret mặc định.

12.2 Nợ kỹ thuật ghi nhận (không làm Tuần 2)

- Access token HS256 dùng chung secret → chuyển ES256 và đưa kycTier vào claim.
- Role DB riêng từng service, REVOKE UPDATE/DELETE trên LedgerEntry.
- Quên PIN, đổi thiết bị, QR login đầy đủ → Tuần 3.
- Gỡ wallet.events, payment.events khi mọi consumer đã chuyển sang finvault.events.
- Redis cho rate limit / nonce khi cần mở rộng.
- Merchant: báo cáo doanh thu, phí merchant, hoàn tiền (REFUNDING, REFUNDED).

13. Phân quyền sở hữu và quy ước làm việc

Mỗi service, bảng và sự kiện có **đúng một chủ sở hữu (R)**. Người khác muốn đổi thì mở PR cho chủ duyệt – kể cả khi chỉ sửa vài dòng.

Thành phần	Chủ (R)	Hỗ trợ	Ghi chú
auth-service (PIN, token, build, secret)	Đạt	Nguyên (FE)	Đạt sửa Dockerfile, compose cho Auth
packages/finvault-internal-auth	Đạt	Khoa	Thêm verifyTransactionToken
gateway (route, chặn Wallet)	Đạt	Khoa	Danh sách route công khai do Đạt duyệt
wallet-service	Khoa	Đạt	Chỉ Khoa đổi schema.prisma của Wallet
payment-service	Khoa	Đạt	Đạt sở hữu requireTransactionToken + bảng jti
user-service (KYC, tier, lookup)	Hữu	Nguyên	Publisher Outbox của user thuộc Hữu
merchant-service	Hữu	Khoa	Ví merchant do consumer Wallet tạo
admin-service (API)	Hữu	Nguyên	Route mới theo mẫu KycService
notification-service	Nguyên	Khoa	Thêm binding finvault.events
packages/finvault-events	Nguyên	Cả nhóm dùng	Xong trước cuối ngày 1
Web: Modal PIN	Đạt	Nguyên	Component dùng chung
Web: P2P, thông báo, Admin KYC, merchant portal	Nguyên	Khoa, Hữu	Khoa, Hữu cung cấp API
docker-compose, init-db.sql, biến môi trường	Đạt	Cả nhóm	Ai thêm biến thì cập nhật .env.example của mình
tests/acceptance	Hữu	Khoa	Nguyên chạy kịch bản trên UI
Tài liệu (README gốc, README Wallet)	Nguyên	Mỗi người phần mình	

13.1 Quy ước chung

- **Nhánh:** `main` luôn chạy được; nhánh `feat/<service>-<việc>`. Mỗi PR cần **ít nhất 1 người review ngoài tác giả**, review trong 24 giờ.
- **Commit:** `type(scope): mô tả`, ví dụ `fix(wallet): lock rows in id order`.
- **Migration:** `YYYYMMDD_<service>_<nội dung>`.
- **Bí mật:** không commit secret thật; xóa mọi giá trị mặc định cứng.
- **Log:** không ghi PIN, token; số CCCD chỉ giữ 4 số cuối.
- **Thời gian:** lưu UTC, hiển thị và tính hạn mức theo giờ Việt Nam (GMT+7).
- **Lỗi API:** mã lỗi chữ hoa (PIN_LOCKED, LIMIT_EXCEEDED…), dạng `{code, message, details?}`.
- **Standup hằng ngày 10 phút**; cuối ngày 4 mỗi người ghi việc chưa xong; ngày 5 đóng băng.